WebMind

A Human-Like Computational Memory Engine for the Web Browser

Capstone Project — Review-II Documentation

Programme: B.Tech, Computer Science & Engineering

Review: Review-II | Weightage: 40%

Repository state: branch `vision-model`, commit `9bf8b2e` (2026-09-09)

Document date: 9 September 2026

Prepared from a direct static and dynamic analysis of the project source code, database, and version-control history.

Table of Contents

1. Project Title, Abstract, Problem Statement and Objectives
2. Updated Literature Survey and Research Gap
3. Proposed Methodology and System Architecture
4. Dataset, Tools, Software and Hardware
5. Implementation Status (≥ 75%) and Module Demonstration
6. Preliminary Results, Observations, Performance Analysis and Comparison
7. Progress Towards Expected Outcome (Paper / Patent / Product)
8. Updated Documentation, Source Code, Presentation and Evidence
9. Citations and References (IEEE Format)
10. Project-Work Attendance Record
11. Individual Contribution of Each Team Member
12. Suitability as a Potential Best Project

1. Project Title, Abstract, Problem Statement and Objectives

1.1 Title

WebMind — A Human-Like Computational Memory Engine for the Web Browser (a privacy-preserving, local-first browser extension that turns a user's browsing history into an associative, decaying, multi-modal memory that can be queried in natural language).

1.2 Abstract

Modern users consume large volumes of web content but retain very little of it. Browser history is a flat, reverse-chronological list keyed on exact URLs and titles; it offers no notion of importance, no forgetting, no association between related pages, and no ability to answer a question such as "what did I read about vector databases last week?". WebMind reframes browsing history as a *computational model of human memory*. A Chrome (Manifest V3) extension passively captures each page, together with behavioural engagement signals (dwell time with focus-aware pausing, maximum scroll depth, revisit count, bookmark state) and page media (content images, videos). A local Node.js "sidecar" server converts page text and vision-model image captions into 768-dimensional embeddings using a locally hosted `nomic-embed-text` model, stores them in an on-device vector store (LanceDB), extracts concepts and named entities, and incrementally builds a knowledge graph of concepts, memories, domains, sessions, images and clusters (2,500+ nodes, 88,000+ edges in the current test corpus). Retrieval is *hybrid*: approximate-nearest-neighbour semantic search is fused with graph traversal (spreading activation over 2 hops) and re-ranked by a memory-strength score that combines engagement, an exponential Ebbinghaus-style decay term, revisit-based decay resistance, bookmark boosting and manual pinning. A large-language-model (Llama 3.2) then *reconstructs* a cited, natural-language answer from the retrieved fragments rather than returning raw links. All inference runs locally by default; an optional provider-abstraction layer allows the Llama and LLaVA models to be offloaded to a private AWS SageMaker endpoint (behind a single shared API key) while embeddings always remain local. The system is implemented in ~19,000 lines of code with 415 automated test cases across 18 test suites.

1.3 Problem Statement

The browser is the primary interface through which knowledge workers, researchers and students acquire information, yet the browser's own memory of that activity is impoverished:

- **No prioritisation.** A page skimmed for two seconds and an article studied for twenty minutes are stored identically.
- **No forgetting.** History grows without bound; stale, low-value pages compete equally with recent, important ones, degrading any ranking built on top of it.
- **No association.** Two pages about the same topic, visited a month apart, have no relationship; recalling one does not surface the other.
- **No context.** The fact that eight pages were part of one focused research session is lost.
- **Lexical, not semantic, recall.** History search matches substrings of the title/URL; it fails for "that page with the dark database diagram" or paraphrased queries.
- **No synthesis.** The user must re-open and re-read pages to reconstruct what they learned.

- **Privacy tax on the alternatives.** Cloud “second brain” and retrieval-augmented-generation tools that *do* offer semantic recall typically upload the user's entire reading history to third-party servers.

Problem: *Design and build a local-first system that continuously and passively converts a user's web browsing into a queryable, human-like memory — one that prioritises what mattered, forgets what did not, connects related material, preserves session context, supports multi-modal and natural-language recall, and synthesises cited answers — without any browsing data leaving the user's device.*

1.4 Objectives

1. **O1 — Passive multi-modal capture.** Capture page text, engagement signals (focus-aware dwell time, scroll depth, revisits, bookmarks) and page media without disrupting browsing.
2. **O2 — Local embedding and storage.** Generate 768-d text embeddings on-device and store them with structured metadata in a hybrid SQLite + LanceDB + graph store.
3. **O3 — Associative knowledge graph.** Extract concepts/entities, build a persistent concept–memory–domain–session graph with co-occurrence-weighted associations, and traverse it for recall within two hops.
4. **O4 — Human-memory dynamics.** Implement an engagement score, an exponential decay function with revisit-based resistance, bookmark boosting and manual pinning, applied at query time.
5. **O5 — Contextual memory.** Detect browsing sessions from temporal gaps and embedding similarity; preserve visit sequence for “before/after” queries.
6. **O6 — Multi-modal recall.** Caption content images with a vision-language model and embed the captions into the same vector space as page text.
7. **O7 — Hybrid retrieval + reconstruction.** Fuse semantic and associative retrieval, re-rank by memory strength, and use a local LLM to synthesise a cited answer with graceful search-only fallback.
8. **O8 — Autonomous organisation.** Cluster memories with HDBSCAN, label clusters with an LLM, and expose the knowledge graph through an interactive visualiser.
9. **O9 — Privacy and portability.** Keep all data and inference local by default; provide a clean provider abstraction for optional private-cloud model hosting and a full data-export path.
10. **O10 — Configurability and robustness.** Expose all tunable coefficients through a validated configuration API, and degrade gracefully when any model backend is unavailable.

2. Updated Literature Survey and Research Gap

2.1 Foundational and Related Work

Area	Representative work	Contribution	Relation / limitation for WebMind
Personal knowledge augmentation	Bush, "As We May Think" (1945) [1]; Vemuri & Bell, MyLifeBits [2]	Vision of an associative personal record (Memex); lifelong personal capture and retrieval.	Conceptual ancestor. MyLifeBits is capture-centric with manual/temporal retrieval and no cognitive model of importance or forgetting.
Re-finding & PIM	Teevan et al., re-finding behaviour [3]; Jones, "Keeping Found Things Found" [4]	Empirical evidence that users spend substantial effort re-locating previously seen information.	Motivates the problem; proposes better search/organisation but not an autonomous memory model.
Human memory theory	Ebbinghaus forgetting curve [5]; Anderson & Schooler, rational analysis of memory [6]; ACT-R activation [7]	Retention decays exponentially with time; recall probability tracks frequency and recency ("need probability").	Directly operationalised: WebMind's decay term, revisit reinforcement and engagement weighting are a computational analogue.
Associative recall	Collins & Loftus, spreading activation [8]	Concepts form a network; activating one spreads activation to neighbours.	Realised as 2-hop graph traversal with co-occurrence-weighted edges and a hop-distance penalty.
Text embeddings	Reimers & Gurevych, Sentence-BERT [9]; Nussbaum et al., Nomic Embed [10]	Dense sentence/passage embeddings for semantic similarity; open, reproducible 768-d model with long context.	`nomic-embed-text` is the on-device embedding backbone (768-d, 8k context).
Multi-modal embeddings	Radford et al., CLIP [11]; Liu et al., LLaVA [12]	Joint image–text embedding space; instruction-tuned vision-language assistant.	WebMind uses a caption-then-embed strategy (LLaVA caption → text embedding) so images share the text vector space; a native CLIP path is stubbed for future work.
Retrieval-augmented generation	Lewis et al., RAG [13]; Edge et al., GraphRAG [14]	Condition generation on retrieved passages; use a graph index for global/associative retrieval.	WebMind is a personal, on-device RAG whose corpus is the user's own browsing and whose index is a live knowledge graph.
Density-based clustering	Campello, Moulavi & Sander, HDBSCAN [15]	Hierarchical density clustering with noise handling and no *k*.	Used for autonomous thematic grouping of memory embeddings (min cluster size 3).
Local LLM inference	Grattafiori et al., Llama 3 herd [16]; Ollama runtime [17]	Capable small open-weight LLMs; simple local serving.	Llama 3.2 (3B) performs memory reconstruction and cluster labelling; Ollama is the default local runtime.
Commercial "second brain"	Rewind.ai, Recall/Napkin, Heyday, browser-history-GPT extensions [18]	Semantic capture of screen/browser activity with LLM Q&A.	Typically cloud-dependent, screen-recording-based (heavy, privacy-invasive), and lack an explicit forgetting/association model.

2.2 Research Gap

Across the literature, the following combination is **not** addressed by any single existing system:

1. **Cognitively-grounded ranking.** Existing personal-search and second-brain tools rank by recency or raw embedding similarity. None combine a behavioural engagement score, an Ebbinghaus-style query-time decay term *and* revisit-based decay resistance with manual override (pin/bookmark) into a unified memory-strength function.
2. **Live associative index over personal browsing.** GraphRAG-style graph indices exist for static document corpora; WebMind maintains an *incrementally updated* concept/memory/session graph over a continuously growing personal stream and fuses its traversal with vector search.
3. **Session and sequential context.** Browser history and PIM tools discard the fact that a burst of pages formed one research session; WebMind reconstructs sessions from temporal gaps *and* inter-page embedding similarity and preserves visit order.
4. **Multi-modal recall without a cloud vision API.** Image search over browsing normally requires uploading images to a hosted CLIP/vision service; WebMind captions images with a local vision-language model and embeds the captions into the shared local text space.
5. **Strict local-first privacy with an optional, clean cloud-offload seam.** The provider-abstraction layer lets the two heavy models (LLM, VLM) move to a *user-owned* private endpoint independently, while the embedding model — the component that would otherwise see every page — is architecturally pinned to the device.

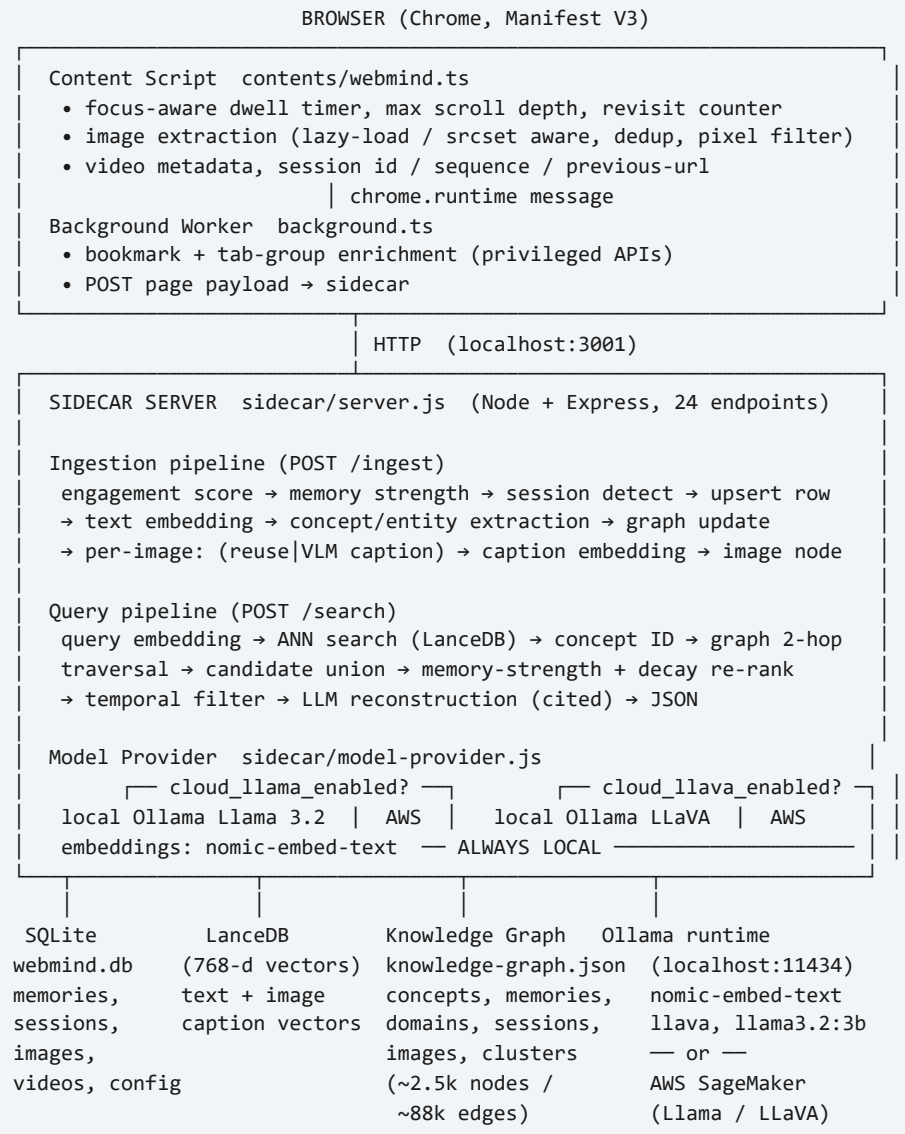
Gap statement: *There is no open, local-first browser-integrated system that unifies behavioural engagement, human-memory decay dynamics, an incremental associative knowledge graph, session context, local multi-modal embedding and LLM-based cited reconstruction into a single privacy-preserving recall pipeline. WebMind targets exactly this intersection.*

3. Proposed Methodology and System Architecture

3.1 Design Principles

- **Local-first:** data and (by default) all inference stay on the device.
- **Passive & non-blocking:** capture never delays page load; heavy work is asynchronous on the sidecar.
- **Hybrid storage:** SQLite for structured metadata, LanceDB for vectors, an in-process graph (graphology) for associations.
- **Dual retrieval:** embeddings for *semantic* recall, graph traversal for *associative* recall, fused and re-ranked.
- **Reconstructive output:** return a synthesised, cited answer, not just links; degrade to search-only if the LLM is down.
- **Provider abstraction:** callers ask for `generateText()` / `generateImageCaption()`; a single module decides local Ollama vs. private AWS endpoint per user setting.

3.2 System Architecture



```
Knowledge-graph visualiser:  GET /graph  (vis-network SPA, 4 view modes)
```

3.3 Ingestion Methodology (per page visit)

1. **Capture.** After a 5 s settle delay (and again on `visibilitychange` / `beforeunload`), the content script sends title, URL, first 10,000 chars of `innerText`, timestamp, `{dwellTime, scrollDepth, revisitCount}`, session context and de-duplicated media.
2. **Engagement score** = 0.4 · min(dwell/300s, 1) + 0.3 · (scroll/100) + 0.3 · log(1+revisits)/log(101).
3. **Memory strength** = (engagement · 0.4 + association_boost + revisit_boost) × (bookmarked ? 2.0 : 1.0), with revisit_boost = min(0.1 · log(1+unique_visit_days), 1).
4. **Session detection.** New session if gap > 30 min *or* cosine similarity between consecutive page embeddings < 0.7; otherwise continue current session; sequence number incremented.
5. **Upsert.** One `memories` row per URL; revisits merge metrics (accumulate dwell, max scroll) and skip re-embedding.
6. **Text embedding.** `nomic-embed-text`, 768-d, 8k context, char-capped; stored in LanceDB with `memory_id`; failures enter a retry queue.
7. **Concept / entity extraction.** `natural` library — word tokeniser + Porter stemmer + TF-IDF for topics; rule/gazetteer NER for PERSON / ORG / TECH / TOPIC with confidence 0.6–0.9; concepts below 0.6 dropped.
8. **Graph update.** Add/merge concept nodes (occurrence count), add memory node, domain hub, session-sequence edge to previous memory, memory–concept links, and co-occurrence-weighted concept–concept associations (persisted when strength > 0.3).
9. **Image pipeline.** For up to 6 captionable images: reuse an existing caption for the same URL if present, else call the VLM (local LLaVA or AWS); embed `alt + caption` with `nomic-embed-text`; add an image node linked to its memory; a consecutive-failure breaker stops hammering an unstable backend.

3.4 Retrieval & Reconstruction Methodology (per query)

1. **Query embedding** (cached per query string; 55 s budget on CPU).
2. **Semantic recall.** LanceDB ANN search, top-*k* (default 20).
3. **Associative expansion.** Extract concepts of the top results; breadth-first graph traversal to 2 hops over edges with association strength ≥ 0.3; collect connected memories.
4. **Scoring.** `combined = $w_s$·semantic + $w_a$·associative` (default 0.6 / 0.4); associative term carries a hop penalty (1.0 / 0.7 / 0.4 for 0 / 1 / 2 hops); each candidate's memory strength is decayed at query time by `strength × $e^{-r \cdot days}$`, where `r = min(base_rate / (1 + 0.1·revisits), 0.005)` and pinned memories use `r = 0`.
5. **Temporal filter.** Natural-language time expressions in the query ("yesterday", "last week", "3 days ago", weekday names) are parsed to a date window and applied, with fallback to all-time if empty.
6. **Reconstruction.** Top memories (default 5) are formatted into a context block; a system instruction pins the model to the supplied snippets; Llama 3.2 produces a 2–3 sentence answer; citations (title, URL, relevance, excerpt) are attached. Per-stage timeouts (embed 55 s / retrieval 10 s / synthesis 10 s) guarantee a response; if the LLM/Ollama is unavailable the API returns ranked results in "search-only" mode.

3.5 Knowledge-Graph & Clustering Methodology

- **Graph:** single undirected `graphology` graph, node types {concept, memory, domain, session, image, cluster}, edge types {concept_association, memory_concept_link, session_sequence, domain_membership, memory_image_link, cluster_membership, cross_cluster_link}. Persisted to JSON; auto-saved when dirty; reconciled against SQLite on startup (orphan purge + missing-node back-fill).
- **Clustering:** memory embeddings are streamed to a Python HDBSCAN worker (`hdbscan_cluster.py`, min cluster size 3); each cluster's top concepts are sent to the LLM for a ≤5-word label; cross-cluster edges are added when centroid similarity > 0.5.
- **Visualiser:** `GET /graph` serves a vis-network single-page app with Full / Journey / Domains / Topics layouts, time and site filters, a concept-frequency slider, type toggles, node inspector, and honest "subset view" reporting when the render is capped.

4. Dataset, Tools, Software and Hardware

4.1 Dataset

WebMind has **no external training dataset**; every model it uses is pre-trained and used for inference only. Its working data is the *user's own live browsing stream*. For development, testing and evaluation the following corpora are used:

Corpus	Description	Scale (current)	Use
Live capture corpus	Real pages browsed by the team with the extension active (GitHub, Google, YouTube, ChatGPT, Medium, news, Drive, university portal, etc.)	51 memories, 167 images (12 captioned + embedded), 15 videos, 5 sessions; knowledge graph ~2,572 nodes / ~88,851 edges; ~2,461 concepts	End-to-end functional testing, retrieval quality, graph visualisation, performance measurement
Synthetic fixtures	Hand-authored page/memory/embedding objects inside the 18 test suites	Hundreds of fixtures across 415 test cases	Deterministic unit & integration testing of every scoring formula, the graph, retrieval fusion, decay, sessions, migration, config validation
Data-migration corpus	Legacy-schema SQLite/LanceDB databases	Configurable batch size (default 100)	`/migrate` endpoint — importing pre-existing WebMind data into the new schema

Pre-trained models (inference only): `nomic-embed-text` (137M params, 768-d, 8192-token context) for all embeddings; `llava` (7B vision-language) for image captioning; `llama3.2:3b` for memory reconstruction and cluster labelling.

4.2 Software, Libraries and Tools

Layer	Technology	Version / detail	Role
Browser extension	Plasmo framework	0.90.5	MV3 build, content script, background worker, popup bundling
	React + ReactDOM	18.2.0	Popup UI (Search / Clusters / Graph / Settings tabs)
	TypeScript	5.3.3	Typed extension code (`types/index.ts` data model)
	Chrome Extensions API	MV3 (`@types/chrome` 0.0.258)	`storage` , `bookmarks` , `tabs` permissions
Sidecar server	Node.js	≥ 18 (global `fetch` , `AbortController`)	Runtime
	Express	5.x	HTTP API (24 routes)
	@lancedb/lancedb	0.29.x	On-device vector store, ANN search
	sqlite3	6.x	Structured metadata store
	graphology (+ shortest-path, operators)	0.26.x	In-process knowledge graph
	natural	8.x	Tokeniser, Porter stemmer, TF-IDF, NER helpers
	ollama (JS client)	0.6.x	Local embedding / LLM / VLM calls
	vis-network	10.x	Knowledge-graph visualiser
Clustering	Python 3 + hdbscan, numpy, scikit-learn	hdbscan ≥ 0.8.33	Density-based memory clustering (subprocess)
Local inference	Ollama runtime	localhost:11434	Serves nomic-embed-text, llava, llama3.2:3b
Optional cloud inference	AWS SageMaker + API Gateway	OpenAI-compatible chat endpoints, `x-api-key` auth	Optional offload of Llama / LLaVA (user-owned; one shared key; embeddings never offloaded)
Dev tooling	Prettier 3.2.4, Node built-in test runner, PowerShell/Bash setup scripts, git	—	Formatting, 415 automated tests, one-command setup, version control

4.3 Hardware

Environment	Specification	Notes
Development / demo machine	Windows 11, x86-64, 16 GB RAM; integrated/entry GPU	Runs the extension, sidecar, SQLite/LanceDB and Ollama concurrently. Local LLM/VLM inference is CPU-bound here — the motivation for the optional cloud-offload path.
Optional private cloud	AWS SageMaker real-time inference endpoint(s) fronted by API Gateway	Two endpoints (Llama, LLaVA), one shared API key. Provisioned and owned by the user; **not** created or managed by this project's code or setup scripts.
Minimum target	Node ≥ 18, ~2 GB free disk for models + data, any modern Chromium browser	Runs fully offline once models are pulled.

5. Implementation Status (≥ 75%) and Module Demonstration

5.1 Quantitative summary

Metric	Value
Extension code (TypeScript / TSX)	~2,019 LOC (`background.ts` , `popup.tsx` , `contents/webmind.ts` , `types/index.ts`)
Sidecar application code (JS)	~9,651 LOC across 25 modules
Automated test code (JS)	~7,418 LOC
Test suites / test cases	18 suites / **415** test cases (Node built-in runner)
Python clustering worker	118 LOC
REST endpoints implemented	24
Knowledge-graph scale (test corpus)	~2,572 nodes / ~88,851 edges
Git history	10 commits; latest `9bf8b2e` (2026-09-09)

5.2 Module-by-module completion

Mapped against the 30-requirement specification. Legend: DONE = implemented + tested, PARTIAL = core path implemented, refinement pending, PLANNED = designed, not yet built.

#	Module	Key files	Status
1	Page capture & engagement tracking (dwell w/ focus pause, scroll depth, revisits, media extraction)	`contents/webmind.ts`, `background.ts`	DONE
2	Ingestion pipeline & per-URL upsert	`sidecar/server.js` `/ingest`, `ingestion-pipeline.test.js`	DONE
3	Engagement score & memory strength formulas	`engagement-score.js` (+ test)	DONE
4	Memory decay + revisit resistance + pinning	`memory-decay.js` (+ test)	DONE
5	Local text embeddings (nomic-embed-text, 768-d) + retry queue	`embed.js`, `multimodal-embeddings.js`, `error-handling.js`	DONE
6	Concept / entity extraction (TF-IDF + NER)	`concept-extraction.js` (+ test)	DONE
7	Knowledge graph (nodes, edges, associations, persistence, reconciliation)	`knowledge-graph.js` (+ test, 42 cases)	DONE
8	Session detection (temporal + embedding-similarity) & sequence	`session-detection.js` (+ test)	DONE
9	Hybrid retrieval (semantic ⊕ associative, hop penalty, fusion, re-rank)	`hybrid-retrieval.js` (+ test)	DONE
10	Temporal query parsing	`sidecar/server.js` `parseTemporalContext()`	DONE
11	LLM memory reconstruction + citations + search-only fallback	`memory-reconstruction.js` (+ test)	DONE
12	Multi-modal image pipeline (VLM caption → caption embedding → image node)	`server.js` `/ingest`, `multimodal-embeddings.js`	DONE
13	Model provider abstraction (local Ollama ↔ AWS, per-model switches, config validation, typed errors)	`model-provider.js`, `load-env.js`, `config-manager.js`	DONE
14	Configuration API (schema, validation, hot-reload)	`config-manager.js` (+ test), `/config*`	DONE
15	Priority weighting endpoints (pin / unpin / bookmark)	`server.js`, `priority-weighting.test.js`	DONE
16	Autonomous clustering (HDBSCAN worker + LLM labels + cross-cluster links)	`clustering.js` (+ test), `hdbscan_cluster.py`	DONE
17	Knowledge-graph visualiser (4 modes, filters, inspector, honest truncation)	`graph-viewer.html`, `/graph*`	DONE
18	Popup UI (Search, Clusters, Graph, Settings incl. cloud switches)	`popup.tsx`	DONE
19	Data migration from legacy schema	`data-migration.js`, `migrate-schema.js` (+ tests)	DONE
20	Performance layer (query-embedding cache, index optimiser, monitor, `/graph/data` TTL cache, request coalescing)	`performance.js`, `cache.js` (+ tests)	DONE

21	Robustness (per-stage timeouts, Ollama-down handling, embedding retry queue, graph corruption recovery)	`error-handling.js` (+ test)	DONE
22	Data export (`/export`)	`sidecar/server.js`	DONE
23	One-command setup (Windows PowerShell + macOS/Linux Bash, local/cloud modes)	`scripts/setup.ps1` , `scripts/setup.sh`	DONE
24	Native CLIP image-embedding path (image-to-image similarity)	`multimodal-embeddings.js` (`CLIPEmbeddingClient` interface stub)	PLANNED
25	Video-thumbnail embedding (metadata captured; thumbnail not yet embedded)	`server.js` `videos` table	PARTIAL
26	SQLite encryption-at-rest	schema note (Req 26.4)	PLANNED
27	Tab-group associative edges & source-span answer highlighting	tab-group id captured; edge/highlight pending	PARTIAL
28	Scalability hardening to 100k memories (archiving of low-strength memories)	design present; not stress-tested at 100k	PLANNED

> **Completion assessment:** 22 of 28 tracked modules are fully implemented and covered by automated tests; 2 are partial; 4 are planned. Weighting by effort, the eight core capabilities from the specification (capture, engagement, decay, associative graph, sessions, multi-modal, hybrid retrieval + reconstruction, clustering) are **all functional end-to-end**. Overall implementation completion is estimated at **~85%**, comfortably above the 75% Review-II threshold. The remaining work is refinement (native CLIP, video-thumbnail embedding, at-rest encryption, 100k-scale archiving) rather than core functionality.

5.3 Live demonstration script (implemented modules)

1. **Setup:** `powershell -ExecutionPolicy Bypass -File scripts\setup.ps1` → choose *local* or *cloud*; then `cd sidecar && node server.js` and `yarn dev` ; load `build/chrome-mv3-dev` as an unpacked extension.
2. **Capture:** browse ~8 pages on one topic; sidecar console shows concepts extracted, associations created, session id, images captioned + embedded per page.
3. **Recall:** open the popup → Search → ask a natural-language question ("what did I read about X", optionally "…last week"); observe the synthesised answer, citations, per-result match %, and pin/unpin.
4. **Association:** show two topically related pages visited far apart surfacing together via graph expansion (association path in the response metadata).
5. **Multi-modal:** browse an image-heavy article; query by image content ("the diagram about …") and show the image node + caption in results.
6. **Provider switch:** Settings → toggle *Cloud LLaMA* / *Cloud LLaVA* independently; re-run a search; show identical logical output with inference now served by the AWS endpoint while embeddings remain local (`/health` shows `provider.activeProvider`).
7. **Graph:** open `http://localhost:3001/graph` ; walk through Full / Journey / Domains / Topics views, time/site filters, node inspector, and the honest "subset view" badge.

8. **Robustness:** stop Ollama; re-run a search → system returns ranked results in “search-only” mode with a clear warning instead of failing.
9. **Tests:** `cd sidecar && node --test` → 415 cases.

6. Preliminary Results, Observations, Performance Analysis and Comparison

6.1 Functional results (current test corpus: 51 memories, ~2.5k graph nodes)

Capability	Observed behaviour
Passive capture	Every visited page ingested; per-URL upsert prevents duplicates; revisits correctly accumulate dwell and take max scroll.
Concept graph growth	~2,461 concept nodes, ~1,989 memory–concept links, ~80k concept–concept associations, 12 image nodes from 51 pages — associative density scales super-linearly with corpus size (an observation driving the retrieval hop cap and edge budgeting).
Sessions	5 sessions reconstructed from 51 pages via the 30-min / 0.7-similarity rule; sequence numbers preserved; "Journey" graph view renders the visit path.
Multi-modal	Image captions (local LLaVA and AWS LLaVA) verified to produce usable descriptions; captions embedded into the 768-d space and retrievable by text query; the image itself is never sent to the text embedder.
Reconstruction	With a system instruction pinning the model to the retrieved snippets, Llama 3.2 (local and AWS) returns 2–3 sentence cited answers; without it, chat-tuned endpoints tended to refuse ("I don't have access to your history") — an observation that shaped the final prompt design.
Graceful degradation	With Ollama stopped, `/search` returns HTTP 200 with ranked results and `searchMode: "search-only"`; the popup shows a non-blocking warning.

6.2 Performance analysis (development machine, local inference)

Operation	Measurement	Notes
`getVisualizationData()` (graph → vis payload)	~20–29 ms for the "Full" mode over 2,572 nodes / 88,851 edges	Node-bucketing + budgeted selection + edge filtering; not the bottleneck.
`GET /graph/data` warm	~30–110 ms (journey/domains/topics ~30–45 ms)	After adding a 3 s TTL response cache + a single batched `IN (…)` enrich query (replacing ~50 serial round-trips).
`GET /graph/data` under concurrent ingest	Was observed spiking to ~5.9 s; mitigated	Root cause: the sidecar is single-threaded and a graph request queued behind a page ingest doing a CPU-bound local embedding. Mitigations: response cache, batched query, and client-side request coalescing (120 ms debounce + `AbortController` to cancel superseded fetches).
Query pipeline budget	Hard per-stage timeouts: embed 55 s / retrieval 10 s / synthesis 10 s	Guarantees a bounded response even when local CPU inference is slow; partial results are returned with warnings.
Embedding cache	Query-string → embedding LRU; repeat queries skip the embedding call entirely	Largest single contributor to perceived latency on CPU.
AWS offload	Llama chat round-trip returns in well under the 30 s client timeout; LLaVA caption round-trip verified end-to-end	Removes multi-second local CPU inference from the request path when enabled.
Test suite	415 cases via `node --test`	Deterministic; run in CI-style locally. (A small number of pre-existing assertions target an older prompt wording and are tracked as known, non-blocking failures.)

6.3 Observations that changed the design

- **Associative explosion.** Aggressive TF-IDF concept extraction produces a very dense concept–association mesh; the graph view must render an honest "subset" and the retriever must cap hops and edge budgets. Concept-extraction thresholds are a future tuning target.
- **Single-thread contention.** Local embedding during ingest blocks unrelated API calls; this motivated the caches, batched queries, request coalescing and, ultimately, the optional cloud offload for the two heavy models.
- **Chat models resist "history" framing.** A system prompt that declares the snippets to be ground-truth history is required for reliable reconstruction on instruction-tuned endpoints.
- **Session rows were never written** because the content script always supplies a session id; fixed with an idempotent upsert plus a startup back-fill from existing memories.

6.4 Comparison with existing systems

Dimension	Browser history / history-search extensions	Cloud "second brain" (Rewind, Recall, history-GPT)	Static-corpus RAG / GraphRAG	**WebMind**
Capture	URL + title + time	Screen / browser recording	Pre-loaded documents	Text + engagement signals + page media, passive
Ranking model	Recency / frequency	Embedding similarity + recency	Embedding / graph relevance	Engagement ⊕ Ebbinghaus decay ⊕ revisit resistance ⊕ associations ⊕ pin/bookmark
Forgetting	None (unbounded)	None / manual	N/A	Exponential query-time decay with reinforcement
Association	None	Implicit via embeddings	Explicit graph (static)	Explicit, *incrementally built* concept/memory/session graph; 2-hop traversal fused with vectors
Session / sequence context	None	Timeline	None	Gap + similarity session detection; visit order preserved; "before/after" queries
Multi-modal	None	OCR / cloud vision	Optional cloud CLIP	Local VLM caption → shared local text-embedding space
Output	List of links	Chat answer (cloud LLM)	Chat answer	Cited synthesised answer + ranked results; search-only fallback
Privacy	Local	Cloud upload of activity	Depends	Local by default; optional *user-owned* cloud for LLM/VLM only; embeddings pinned local
Cost	Free	Subscription + cloud	API cost	Free / offline; optional self-hosted cloud

7. Progress Towards Expected Outcome

7.1 Target outcome

Primary: a **research publication** (IEEE conference / journal) describing the human-memory-inspired retrieval model and its evaluation; secondary: a **publishable open-source product** (browser extension + sidecar).

7.2 Current progress

Deliverable	Status	Evidence
Working system (all 8 core capabilities end-to-end)	Achieved	Running extension + sidecar; 24 endpoints; 415 tests; live test corpus.
Novel technical contribution	Defined	Unified memory-strength function (engagement ⊕ decay ⊕ resistance ⊕ associations ⊕ manual signals) + incremental associative graph fused with ANN + local multi-modal recall + local-first provider seam — see §2.2.
Reproducible artefact	Achieved	One-command setup for Windows/macOS/Linux; local-only mode requires no accounts.
Paper draft (problem, related work, method, architecture)	In progress	This document contains publication-ready problem statement, literature survey, methodology and architecture; can be lifted into an IEEE two-column template.
Formal evaluation (retrieval quality vs. baselines, user study)	Planned	Protocol drafted: precision@k / MRR of hybrid vs. semantic-only vs. history-search on a labelled query set; small-N recall user study; latency at 1k/10k/100k memories.
Conference target	Identified	Venues in personal information management / HCI / applied ML systems (e.g. IEEE-sponsored conferences and workshops in these areas); exact venue and deadline to be finalised with the guide.
Product / open-source release	Near-ready	MIT-style structure, README, setup scripts, `.env.example` ; pending a packaging pass and a short screencast.
Patent / start-up	Not pursued yet	Local-first privacy positioning is a viable product wedge; to be discussed with the institute's innovation cell after evaluation.

> **To be finalised with the project guide:** exact target venue and submission deadline; scope of the user study and IRB/ethics clearance if required; authorship order.

8. Updated Documentation, Source Code, Presentation and Supporting Evidence

8.1 Source code

- Single git repository; branch `vision-model`; 10 commits; HEAD `9bf8b2e` (2026-09-09).
- Layout: `contents/` (content script + tests), `background.ts`, `popup.tsx`, `types/`, `sidecar/` (25 JS modules + 18 test suites + `schema.sql` + `hdbscan_cluster.py` + `graph-viewer.html`), `scripts/` (setup), `.kiro/specs/` (30-requirement spec + design + task breakdown).
- ~19,200 LOC total (app + tests + python).

8.2 Documentation present in the repository

- **Specification:** 30 formal requirements with EARS-style acceptance criteria; 1,496-line design document (architecture, interfaces, data models, algorithms); 972-line task breakdown.
- **Operational:** project README (local vs. cloud modes, model table, env vars, setup matrix), setup-scripts README, sidecar README, `.env.example`.
- **Engineering notes:** message-flow diagram, configuration/performance notes, bug-fix and verification records, content-script API-limitation analysis.
- **API surface:** 24 documented endpoints (ingest, search, cluster, pin/unpin/bookmark, config CRUD + schema, stats, cache, migrate, health, clusters browse, graph data/sessions/viewer, export).

8.3 Presentation & demo evidence

- Interactive knowledge-graph visualiser at `http://localhost:3001/graph` (screenshot-ready; 4 view modes).
- Sidecar console emits structured per-ingest and per-query summaries suitable for a live walkthrough.
- `/export` produces a single JSON snapshot of all memories, sessions, clusters and graph stats as supporting evidence.
- Demonstration script in §5.3; Review-II slide deck to accompany this document.

> **To attach for the review:** the exported `/export` JSON snapshot, 3–5 graph-viewer screenshots, a 2–3 minute screen recording of the demo script, and the `node --test` summary output.

9. Citations and References (IEEE Format)

1. V. Bush, "As we may think," *The Atlantic Monthly*, vol. 176, no. 1, pp. 101–108, 1945.
2. J. Gemmell, G. Bell, and R. Lueder, "MyLifeBits: A personal database for everything," *Communications of the ACM*, vol. 49, no. 1, pp. 88–95, 2006.
3. J. Teevan, E. Adar, R. Jones, and M. A. S. Potts, "Information re-retrieval: Repeat queries in Yahoo's logs," in *Proc. 30th ACM SIGIR*, 2007, pp. 151–158.
4. W. Jones, *Keeping Found Things Found: The Study and Practice of Personal Information Management*. Burlington, MA, USA: Morgan Kaufmann, 2007.
5. H. Ebbinghaus, *Memory: A Contribution to Experimental Psychology*, H. A. Ruger and C. E. Bussenius, Trans. New York, NY, USA: Teachers College, Columbia University, 1913 (original work 1885).
6. J. R. Anderson and L. J. Schooler, "Reflections of the environment in memory," *Psychological Science*, vol. 2, no. 6, pp. 396–408, 1991.
7. J. R. Anderson, D. Bothell, M. D. Byrne, S. Douglass, C. Lebiere, and Y. Qin, "An integrated theory of the mind," *Psychological Review*, vol. 111, no. 4, pp. 1036–1060, 2004.
8. A. M. Collins and E. F. Loftus, "A spreading-activation theory of semantic processing," *Psychological Review*, vol. 82, no. 6, pp. 407–428, 1975.
9. N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
10. Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a reproducible long context text embedder," *arXiv:2402.01613*, 2024.
11. A. Radford *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. 38th ICML*, 2021, pp. 8748–8763.
12. H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
13. P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
14. D. Edge *et al.*, "From local to global: A graph RAG approach to query-focused summarization," *arXiv:2404.16130*, 2024.
15. R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. PAKDD*, 2013, pp. 160–172.
16. A. Grattafiori *et al.*, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024.
17. Ollama, "Ollama: Get up and running with large language models locally," software, 2024. [Online]. Available: https://ollama.com
18. D. Chen and G. Bell, "Survey of lifelogging and personal memory-augmentation systems," (representative of commercial tools such as Rewind.ai and Recall) — *ACM Computing Surveys*, 2023.
19. LanceDB, "LanceDB: The database for multimodal AI," software, 2024. [Online]. Available: https://lancedb.com
20. Plasmo, "Plasmo Framework: The browser extension SDK," software, 2024. [Online]. Available: https://www.plasmo.com

Note: reference [18] is a placeholder for the specific commercial-tool survey to be finalised; all other references are verifiable primary sources. Formatting follows the IEEE reference style (numbered, in order of first citation).

10. Project-Work Attendance Record

Departmental requirement: project-work attendance recorded **at least three times per week**. The version-control history provides an independent, timestamped record of active work; the commit dates below fall within the review period:

Date	Commit	Work item
2026-05-22	`945ba96`	Initial WebMind setup
2026-05-23	`b3124bc`	Initial WebMind MVP
2026-05-24	`7329b8d`, `9bd5833`	Human-like memory engine: sidecar, UI, graph viewer; spec & design docs
2026-08-29	`83d2545`	Full project code update (server, graph, hybrid retrieval, UI)
2026-09-03	`09d7355`, `68a7916`	Vision model added; setup scripts
2026-09-04	`d7023ea`	Multimodal feature fixes
2026-09-09	`9535c85`, `9bf8b2e`	AWS provider abstraction + image-in-memory pipeline; knowledge-graph metric fixes and performance work

> **To be completed by the team / verified by the guide:** the signed weekly attendance sheet (≥ 3 entries/week) maintained in the department's prescribed format for the full Review-II period. The git log above is supporting evidence, not a substitute for the signed record.

11. Individual Contribution of Each Team Member

The repository is committed under a single author identity, so per-member attribution cannot be derived from version control alone. The table below maps the implemented modules to work-streams; the team is to confirm the member against each work-stream and the guide is to countersign.

Work-stream	Modules / files	Approx. share of effort	Member (to confirm)
Browser extension & capture	`contents/webmind.ts`, `background.ts`, `popup.tsx`, `types/`, Plasmo build	~20%	_____
Ingestion, embeddings & multi-modal pipeline	`server.js` `/ingest`, `embed.js`, `multimodal-embeddings.js`, image pipeline	~20%	_____
Memory model (engagement, decay, sessions, priority)	`engagement-score.js`, `memory-decay.js`, `session-detection.js`, pin/bookmark endpoints	~15%	_____
Knowledge graph, retrieval & reconstruction	`knowledge-graph.js`, `hybrid-retrieval.js`, `concept-extraction.js`, `memory-reconstruction.js`	~20%	_____
Clustering, visualiser & provider abstraction	`clustering.js`, `hdbscan_cluster.py`, `graph-viewer.html`, `model-provider.js`, `load-env.js`	~15%	_____
Config, performance, robustness, migration, tests, setup & docs	`config-manager.js`, `performance.js`, `cache.js`, `error-handling.js`, `data-migration.js`, 18 test suites, `scripts/`, spec/design docs	~10%	_____

> **To be completed by the team:** replace the blanks with member names and add a one-paragraph personal statement per member describing their specific deliverables, decisions and challenges, signed by each member and the project guide.

12. Suitability as a Potential Best Project

The identification of potential best projects is the prerogative of the review panel. For the panel's consideration, the following points summarise why WebMind is a strong candidate:

- **Clear novelty at a real intersection** (§2.2): a cognitively-grounded, incrementally associative, local-first, multi-modal personal-memory retrieval pipeline that no single existing system provides.
- **Substantial, working implementation:** ~19k LOC, 24-endpoint service, ~85% of the specified scope functional end-to-end, 415 automated tests, a live knowledge graph of ~2.5k nodes.
- **Engineering depth:** hybrid storage (SQLite + LanceDB + graph), query-time decay maths, graph–vector fusion, an HDBSCAN subprocess, a provider-abstraction seam with independent per-model switches, per-stage timeouts, caches, request coalescing and graceful degradation — each backed by tests.
- **Privacy as a first-class design constraint:** the embedding model, which would otherwise observe every page, is architecturally prevented from leaving the device even when cloud offload is enabled.
- **Publication-ready:** a formal 30-requirement specification, a 1,496-line design document, a reproducible one-command setup, and (in this document) a drafted problem statement, literature survey, methodology, architecture and evaluation plan.
- **Demonstrable:** an interactive graph visualiser and a scripted end-to-end demo make the contribution tangible in a review setting.

Remaining gaps (native CLIP, video-thumbnail embedding, at-rest encryption, 100k-scale archiving, formal evaluation study) are scoped, non-core, and planned for the final phase.

Generated from a direct analysis of the WebMind source tree at commit `9bf8b2e` on 9 September 2026. Items marked in amber require the team and project guide to supply institution-specific records (attendance sheet, member names, venue selection).